AI SMART ALARM

PYTHON FOR AI PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF THE DEGREE OF **BACHELOR OF TECHNOLOGY** IN **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** OF THE ANNA UNIVERSITY

PYTHON FOR AI PROJECT
APRIL/MAY 2026

PROJECT WORK

Submitted by

NAVEEN KUMAR M -722825243127

BATCH 2025 – 2029

Under the Guidance of

Mr.R.Divakar ,M.E., (Ph.D)

Assistant Professor

Department of Artificial Intelligence and Data Science

Sri Eshwar College of Engineering

(An Autonomous Institution – Affiliated to Anna University)

COIMBATORE – 641 202

Sri Eshwar College of Engineering

(An Autonomous Institution – Affiliated to Anna University)
COIMBATORE – 641 202

BONAFIDE CERTIFICATE

Certified that this Report titled **" AI SMART ALARM "** is the bonafide work of **NAVEEN KUMAR M - 722825243127** who carried out the project work under my supervision.

SIGNATURE

Dr. R. Suresh, M.Sc., Ph.D**. HEAD OF THE DEPARTMENT**

Science and Humanities,

Sri Eshwar College of Engineering,

Coimbatore – 641 202.

SIGNATURE

Mr.R.Divakar M.E., (Ph.D)

SUPERVISOR

Assistant Professor,

Department Of Artificial Intelligence and Data Science,

Sri Eshwar College of Engineering,
Coimbatore – 641 202.

Submitted for the **Autonomous End Semester Project Viva-Voce** held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

Sri Eshwar College of Engineering
Coimbatore | Tamilnadu
An Autonomous Institution
Affiliated to Anna University, Chennai

QUALITY POLICY

To establish a system of Quality Enhancement, which would on a continuous basis evaluate and enhance the quality of teaching – learning, research and extension activities of the institution, leading to improvements in all processes, enabling the institution to attain excellence.

INSTITUTE VISION

To be recognized as a premier institution, grooming students into globally acknowledged engineering professionals.

INSTITUTE MISSION

- Providing outcome and value-based engineering education
- Nurturing research and entrepreneurial culture
- Enabling students to be industry ready and fulfill their career aspirations
- Grooming students through behavioral and leadership training programs
- Making students socially responsible

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

DEPARTMENT VISION

To build a conducive academic and research environment in the stream of Artificial Intelligence and Data Science for enabling global education, research and entrepreneurship.

DEPARTMENT MISSION

- Update and enrich the curriculum by integrating global trends and technological advancements to ensure the academic excellence.
- Promote a research-driven environment by providing access to state-of-the-art infrastructure with emerging technologies.
- Empower the faculty through continuous upskilling, international collaborations, and active research engagement to provide globally competent education.

- Encourage student-centric, industry-aligned learning that nurtures innovation and entrepreneurship activities for solving real-world and societal challenges.

PROGRAM EDUCATIONAL OBJECTIVES

Graduates will be able to:

PEO1: Build a progressive professional career in industry, or pursue higher education, or
succeed as an entrepreneur.

PEO2: Excel in cutting-edge technologies of Artificial Intelligence and Data Science to address real world challenge

PEO3: Exhibit professional ethics, involvement in team work in their profession and contributing to the advancement of society.

PROGRAM OUTCOMES

PO1 - Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 - Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 - Design/development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 - Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 - Modern Tool Usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6 - The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 - Environment and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8 - Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9 - Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings

PO10 - Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 - Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multi disciplinary environments.

PO12 - Life-long Learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSO1. Apply programming, problem-solving, and hardware integration skills with AI and
data science to design intelligent, secure and efficient systems.

PSO2. Develop an AI-driven application by exploring optimized coding competencies using predictive modeling and data analytics technologies.

DECLARATION

NAVEEN KUMAR M [722825243127]

To declare that the project entitled **"AI SMART ALARM "** submitted in partial fulfilment to the University as the project work of Bachelor of Technology (Artificial Intelligence and Data Science) Degree, is a record of original work done by me under the supervision and guidance of **Mr.R.Divakar M.E., (Ph.D)** Assistant Professor, Department of Artificial Intelligence and Data Science, Sri Eshwar College of Engineering, Coimbatore.

Place: Coimbatore

Date:

[NAVEEN KUMAR M]

Project Guided by,
Mr. R.Divakar M.E., (Ph.D)
Assistant Professor / AI-DS

ACKNOWLEDGEMENT

The success of a work depends on a team and cooperation. I take this opportunity to express my gratitude and thanks to everyone who helped me with my project. I would like to thank the management for the constant support provided by them to complete this project.

It is indeed our great honor bounded duty to thank our beloved **Chairman, Mr. R. Mohanram,** for his academic interest shown towards the students.

We are indebted to our **Director, Mr. R. Rajaram,** for motivating and providing us with all facilities.

I wish to express my sincere regards and deep sense of gratitude to **Dr. Sudha Mohanram, M.E, PhD. Principal**, for the excellent facilities and encouragement provided during the course of the study and project.

We are indebted to **Dr. R. Suresh, M.Sc., Ph.D.** Head of Science and Humanities

Department for having permitted us to carry out this project and giving the complete freedom to utilize the resources of the department

I express my sincere thanks to my guide **Mr.R.Divakar M.E., (Ph.D)** Assistant Professor of Artificial Intelligence and Data Science Department for the valuable guidance and encouragement given to me for completing this project. I solemnly express our thanks to all the teaching and nonteaching staff of the

Artificial Intelligence and Data Science Department, family and friends for their valuable support, which inspired me to work on this project.

TABLE OF CONTENTS

CHAPTER	CONTENT		PAGE NO
	LIST OF FIGURES		1
	ABSTRACT		2
1	**INTRODUCTON**		
	1.1	BACKGROUND AND MOTIVATION	3
	1.2	PROBLEM STATEMENT AND OBJECTIVES	3
2	**SYSTEM REQUIREMENTS & SPECIFICATION**		
	2.1	TECHNICAL ENVIRONMENT	4
	2.2	FUNCTIONAL & NON-FUNCTIONAL REQUIREMENTS	5
3	**LOW-LEVEL DESIGN (LLD)**		
	3.1	MODULE/COMPONENT OVERVIEW	6
	3.2	MODULE AND DESIGN	7
4	**DATABASE DESIGN**		
	4.1	ENTITY-RELATIONSHIP MODEL	11
	4.2	SCHEMA DETAILS	12
5	**IMPLEMENTATION AND RESULT**		
	5.1	SYSTEM IMPLEMENTATION	13
	5.2	OUTPUT	19
6	**CONCLUSION AND FUTURE SCOPE**		
	6.1	FUTURE ENHANCEMENTS AND MAINTENANCE	24
7	**APPENDICES AND REFERENCES**		
	7.1	REFERENCES	25
	7.2	APPENDIX A	26

LIST OF FIGURES

FIGURE NO	LIST OF FIGURES	PAGE NO
3.1	FLOWCHART	**9**
3.2	USE CASE DIAGRAM	**10**
5.1	TIME SET PAGE	**19**
5.2	ALARM SET PAGE	**19**
5.3	TIME TO RING PAGE	**20**
5.4	ALARM RINGINING PAGE	**20**

ABSTRACT

The **Smart Alarm AI** is an intelligent web-based application designed to optimize users' sleep schedules by predicting ideal wake-up times based on natural sleep cycles. The system is built using the Flask framework, providing a lightweight and efficient backend integrated with a responsive HTML, CSS, and JavaScript frontend.

The application utilizes scientifically proven 90-minute REM sleep cycles to calculate optimal wake-up windows. Based on user input, the system generates multiple wake-up time suggestions categorized into **Average, Good, and Excellent** sleep quality levels, each with percentage-based scoring to enhance user understanding.

The system also includes a **history tracking feature**, allowing users to review previous sleep predictions. Designed with a stateless architecture, all computations are performed in-memory without persistent data storage, ensuring simplicity and user privacy.

The result is a user-friendly, efficient, and intelligent solution that helps individuals improve productivity, health, and daily performance by aligning wake-up times with optimal sleep cycles

INTRODUCTION

1.1 Background and Motivation

In today's fast-paced digital world, irregular sleep patterns and poor time management significantly affect productivity and health. Many individuals rely on traditional alarms that interrupt sleep cycles, causing fatigue and reduced efficiency.

The Smart Alarm AI addresses this issue by leveraging sleep science principles, specifically the 90-minute REM cycle, to determine the best wake-up times. Instead of waking users abruptly, the system ensures they wake up at the end of a sleep cycle, resulting in better alertness and improved well-being

1.2 Problem Statement and Objectives

Traditional alarm systems do not consider biological sleep cycles, leading to inefficient wake-up experiences. This project aims to develop an intelligent web-based system that predicts optimal wake-up times.

The key objectives of this project are:

- Develop a responsive web interface for user input
- Implement sleep cycle prediction using Python
- Classify wake-up times into quality levels (Average, Good, Excellent)
- Provide percentage-based evaluation
- Implement history tracking for user reference
- Ensure stateless architecture for privacy

2.1. Technical Environment

This section outlines the essential hardware and software requirements for the successful deployment and operation of the Smart Alarm AI – Sleep Cycle Prediction System.

Hardware Requirements:

- **Processor:** Intel Core i3/i5 or AMD equivalent (Dual-core 2.0 GHz minimum)
- **RAM**: 4 GB or more (8 GB recommended for smooth performance)
- **Storage**: Minimum 1 GB free space
- **Network**: Basic internet connection (only required for browser access, no external APIs used)

Software/OS Prerequisites:

- **Backend**: Python 3.10+ with Flask Framework
- **Frontend**: HTML5, CSS3, JavaScript
- **Libraries Used:**
  - datetime (for time calculations)
  - Flask (for web framework)
- **Development Tools:**
  - Visual Studio Code (IDE)
  - Web Browser (Chrome/Edge/Firefox)
- **Operating System:**
  - Windows 10/11, macOS, or Linux

2.2. Functional & Non-Functional Requirements

Expected Functionalities (Functional):

The system allows users to input their sleep time through a simple web interface. Based on this input, the application calculates optimal wake-up times using 90-minute sleep cyclesExpected Functionalities:

The system:

- Accepts user sleep time input
- Predicts multiple wake-up time options
- Classifies results into Average, Good, and Excellent sleep quality
- Displays percentage-based sleep quality
- Provides an interactive visual interface with progress indicators
- Maintains a temporary history of previous predictions
- Allows users to clear history

Performance (Non-Functional):

The application is designed to be lightweight and efficient, as it does not rely on external APIs or heavy computations.

- Fast response time (instant prediction using Python logic)
- Minimal resource usage (no database or external API calls)
- Smooth UI rendering with optimized frontend design
- Supports real-time interaction without delays

Security & Privacy (Non-Functional):

The system follows a stateless architecture, ensuring user privacy and simplicity.

- No personal data is stored permanently
- All computations are performed in-memory
- Data is cleared upon page refresh or server restart
- No external API usage, reducing security risks

3.1 Module/Component Overview:

The **Smart Alarm AI** follows a simple, layered architecture to ensure clear separation of concerns between the user interface, backend processing, and prediction logic.Captures voice input from the user through a microphone. It ensures proper audio acquisition and forwards the input for further processing.

- **User Interface (Frontend) :**Built using **HTML, CSS, and JavaScript**, the frontend handles:

Flask Routes (Controllers):

- **/anlyze:** Handles both GET and POST requests. Accepts user sleep time input Calls prediction logic Sends results to frontend
- **/clear :**Clears stored history Resets application state
- **External Services Layer**: The core "brain" of the app. It relies on the Sightengine API for computer vision tasks and the Google Gemini SDK (with Google Search tools) for natural language reasoning.

3.2 Module and Design

The Smart Alarm AI system is designed using a modular architecture to ensure simplicity, clarity, and efficient processing. The system is divided into multiple functional modules, each responsible for a specific task in the overall workflow.

- **User Input Module**

  This module handles user interaction through the web interface. It allows users to input their sleep time using a time selection field. The module ensures that the input is collected in a proper format (HH:MM) before sending it to the backend for processing.

- **Validation Module**

  The validation module checks whether the user input is valid and correctly formatted. If the input is missing or incorrect, the system displays an appropriate error message and prevents further processing. This ensures data accuracy and system reliability.

- **Processing Module**

  This module converts the user input into a datetime format and prepares it for further computation. It acts as a bridge between user input and the prediction logic, ensuring that the data is processed correctly.

- **Sleep Cycle Prediction Module**

  This is the core module of the system. It calculates optimal wake-up times based on 90-minute sleep cycles. The system considers multiple cycles (4, 5, and 6 cycles) to generate different wake-up options for the user.

- **Quality Analysis Module**

  After calculating wake-up times, this module assigns a quality level to each result. The categories include:

  - Average (4 cycles)
  - Good (5 cycles)
  - Excellent (6 cycles)

  Each category is also associated with a percentage value to provide better user understanding.

- **Result Display Module**

  This module presents the calculated wake-up times, quality levels, and percentages to the user through the web interface. It uses visual elements such as cards and progress bars to enhance readability and user experience.

- **Alarm Module**

  The alarm module is responsible for triggering alerts at the selected wake-up time. It includes features such as sound alerts, visual indicators, and interactive mechanisms like question-based alarm dismissal to ensure user engagement.

- **History Management Module**

  This module stores the user's previous inputs and results temporarily in memory. It allows users to view past sleep predictions and provides an option to clear the history when needed

FLOW CHART:

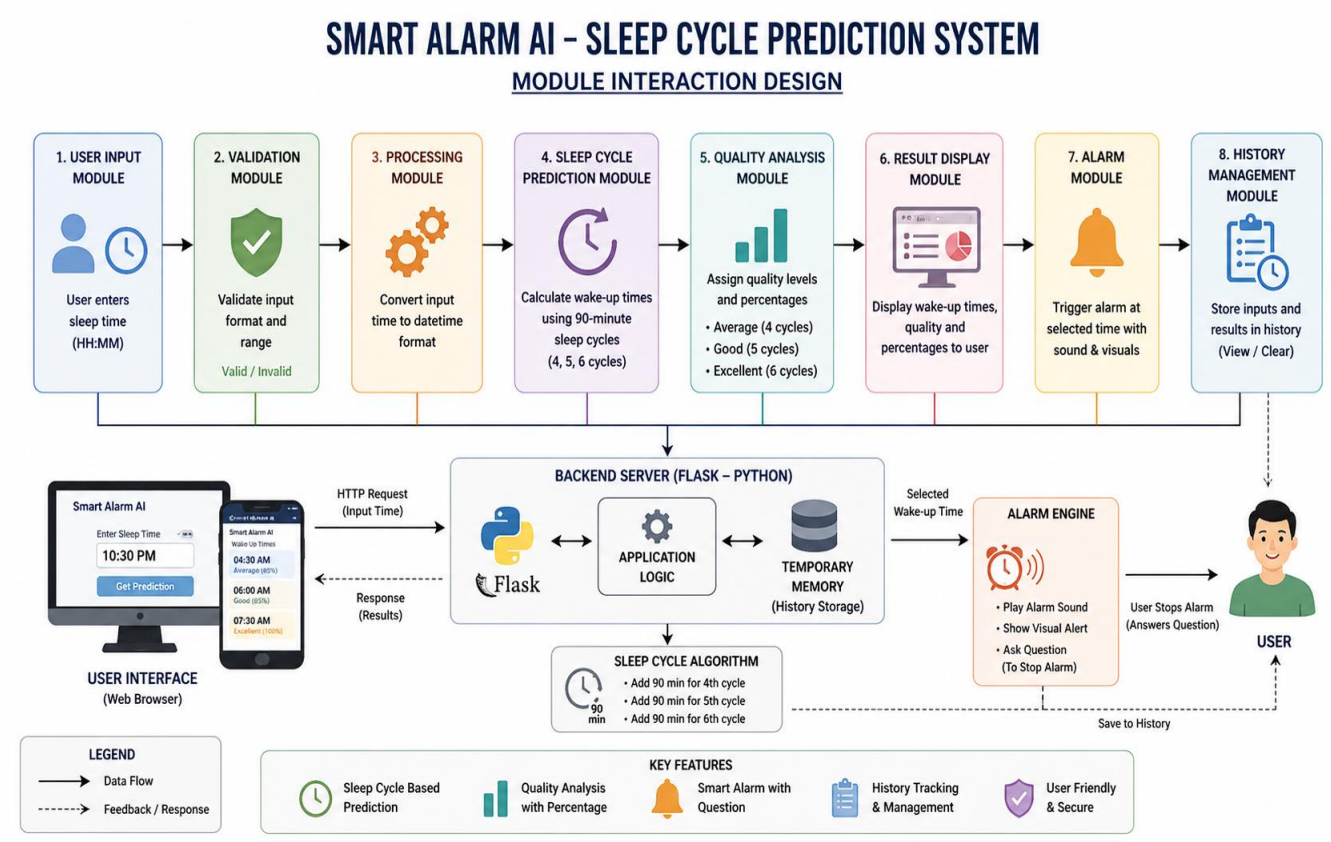

Fig:3.1.Flow Chart

- **User Input:** The system allows users to enter their sleep time through a web interface, which is captured in a standard time format (HH:MM).
- **Input Processing:** The entered time is processed using Python's datetime module to convert it into a suitable format for calculation.
- **Sleep Cycle Prediction:** The system applies 90-minute sleep cycle logic to compute multiple optimal wake-up times (4, 5, and 6 cycles).
- **Quality Analysis**: Each predicted wake-up time is evaluated and categorized into Average, Good, and Excellent, along with percentage-based indicators.
- **Data Handling:** A Flask-based backend processes the input and temporarily stores results in memory for history tracking.
- **Alarm Execution:** The selected wake-up time triggers an alarm with sound, visual alerts, and an interactive mechanism (question-based stopping).

USE CASE DIAGRAM

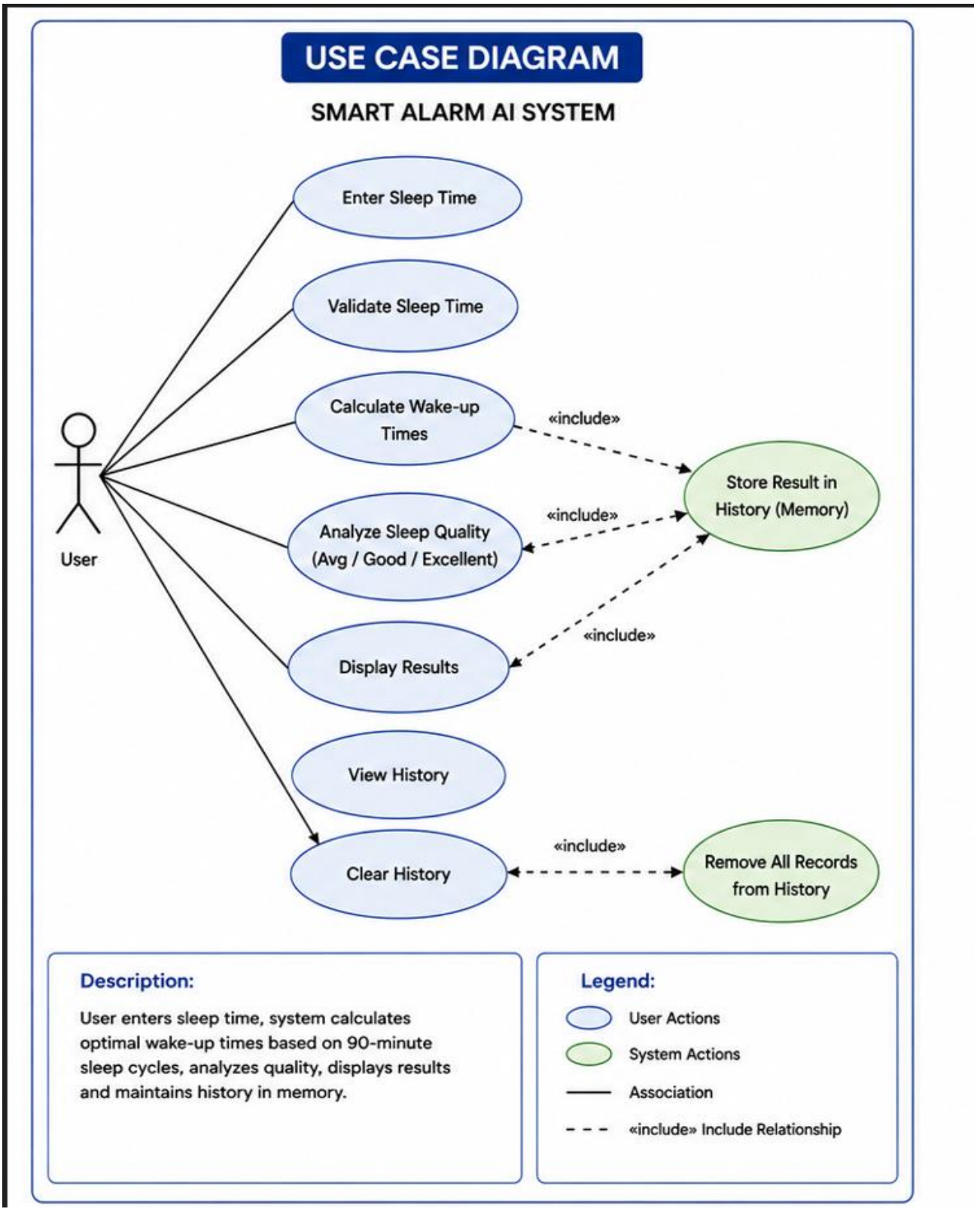

Fig:3.2.Use Case Diagram

4.1. ENTITY-RELATIONSHIP MODEL

The **Smart Alarm AI – Sleep Cycle Prediction System** does not rely on a traditional database management system. Instead, it uses **in-memory data handling within Python (Flask)** to manage user input, processed results, and temporary history storage. The relationships between data are maintained using variables, lists, and dictionaries during runtime.

User Input Data

- **Primary Key:** input_id
- **Key Attributes:** sleep_time, input_format, timestamp

Processed Sleep Data

- **Primary Key:** process_id
- **Key Attributes:** sleep_time_converted, datetime_format

Wake-Up Prediction Data

- **Primary Key:** prediction_id
- **Key Attributes:** wake_time, cycle_count, total_sleep_duration

Sleep Quality Data

- **Primary Key:** quality_id
- **Key Attributes:** quality_level (Average/Good/Excellent), percentage

4.2. SCHEMA DETAILS

If future enhancements include user accounts and long-term tracking, a database system can be integrated.

Proposed Database (MongoDB / SQL):

Users Collection:

- _id
- username
- email
- password (hashed)

Sleep History Collection:

- _id
- user_id (Reference)
- sleep_time
- predicted_wake_times
- quality_level
- percentage_score
- timestamp

5.1 SYSTEM IMPLEMENTATION

The **Smart Alarm AI – Sleep Cycle Prediction System** is implemented as a web-based application using Python and the Flask framework.

5.2 Backend Implementation

The backend of the system is developed using **Python with the Flask framework**. It is responsible for handling user requests, processing input data, and generating results.

- The Flask server receives user input (sleep time) through HTTP POST requests.
- The input is processed using Python's **datetime module** to perform time-based calculations.
- The system computes multiple wake-up times using **90-minute sleep cycles (4, 5, 6 cycles)**.
- Results are formatted and sent back to the frontend for display.
- Temporary data such as user history is stored using in-memory structures (lists/dictionaries).

5.3 Frontend Implementation

The frontend is designed using HTML, CSS, and JavaScript to create a user-friendly interface.

- HTML is used to structure the webpage and input forms.
- CSS provides styling and responsive design for better user experience.
- JavaScript enables dynamic interaction such as form handling, countdown timers, and alarm triggering.
- The interface displays wake-up times using cards and progress indicators.

5.4 Sleep Cycle Algorithm Implementation

- **The system uses a simple algorithm based on sleep cycle theory:**
- **Each sleep cycle is assumed to be 90 minutes.**
- **The system calculates wake-up times by adding multiples of 90 minutes to the user's sleep time.**
- **Three main cycles are considered:**
    - **cycles → Average**
    - **cycles → Good**
    - **cycles → Excellent**

5.5 Alarm System Implementation

The alarm module is implemented using JavaScript and browser-based features.

- The system triggers an alarm when the selected wake-up time is reached.
- Alarm features include:
    - Sound alerts using Web Audio API
    - Visual alerts (screen flashing / messages)
    - Interactive stop mechanism (user must answer a question)

5.6 OUTPUT

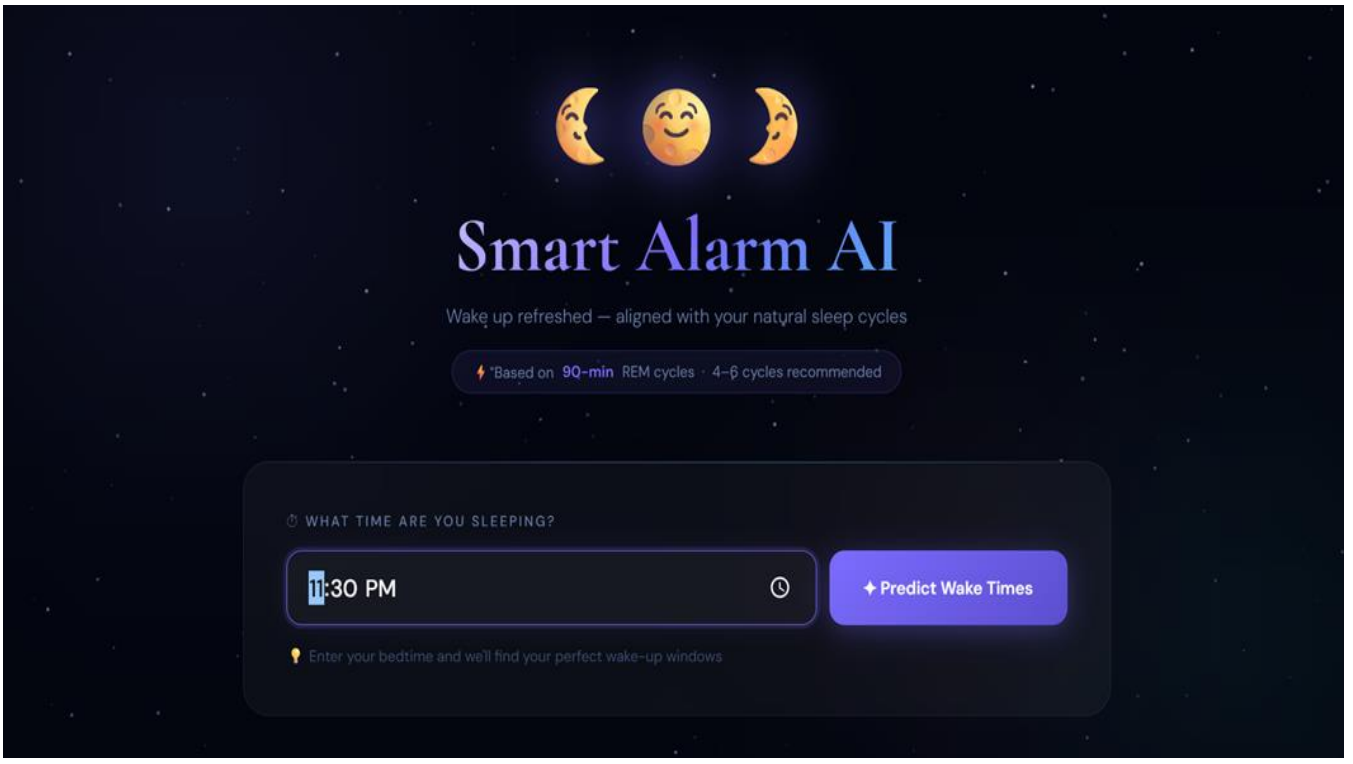

Fig:5.1. Time set Page

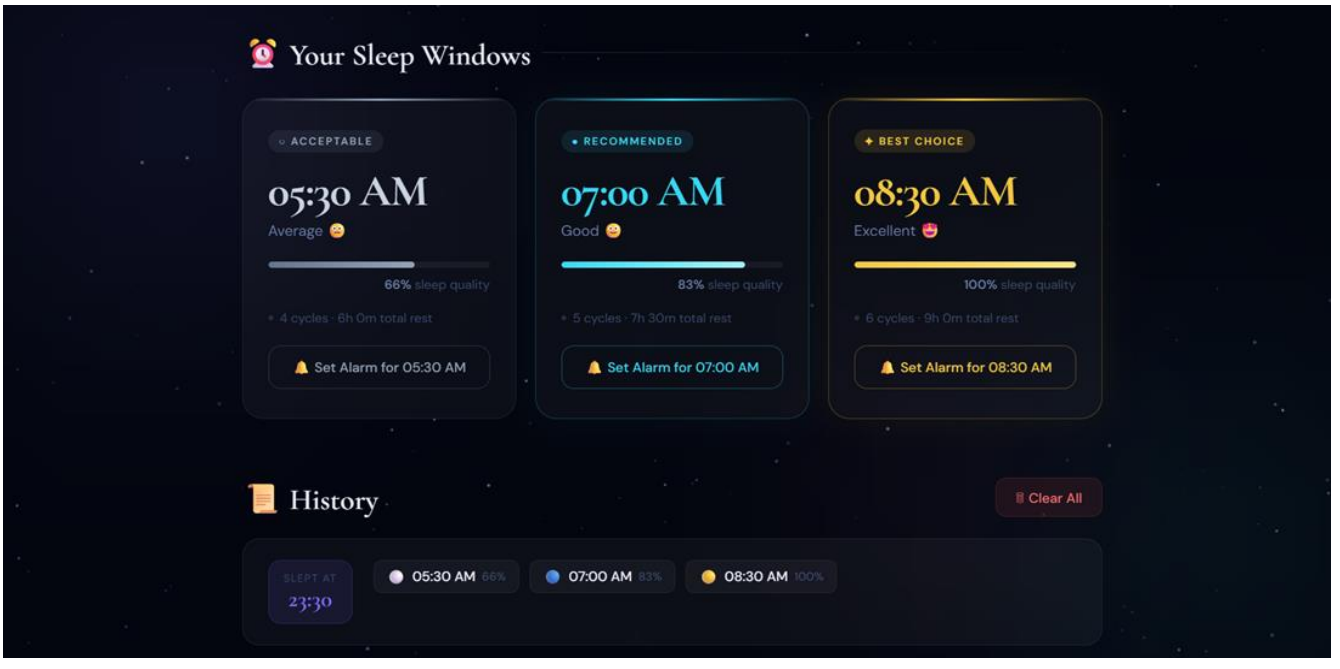

Fig:5.2. Set Alarm Page

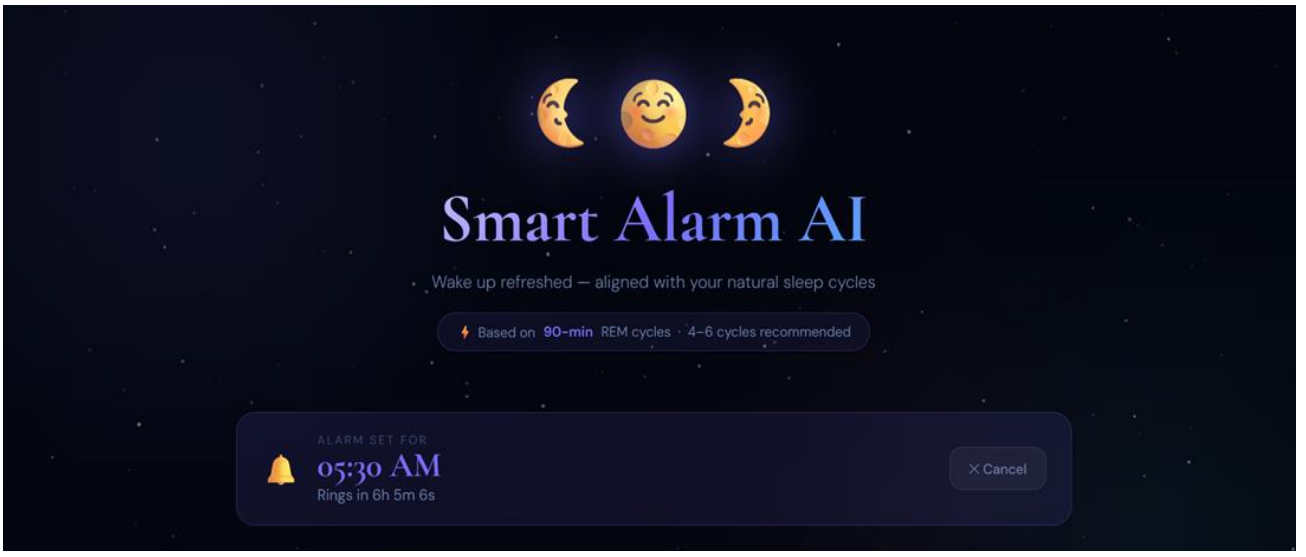

Fig:5.3. Time to Ring Page

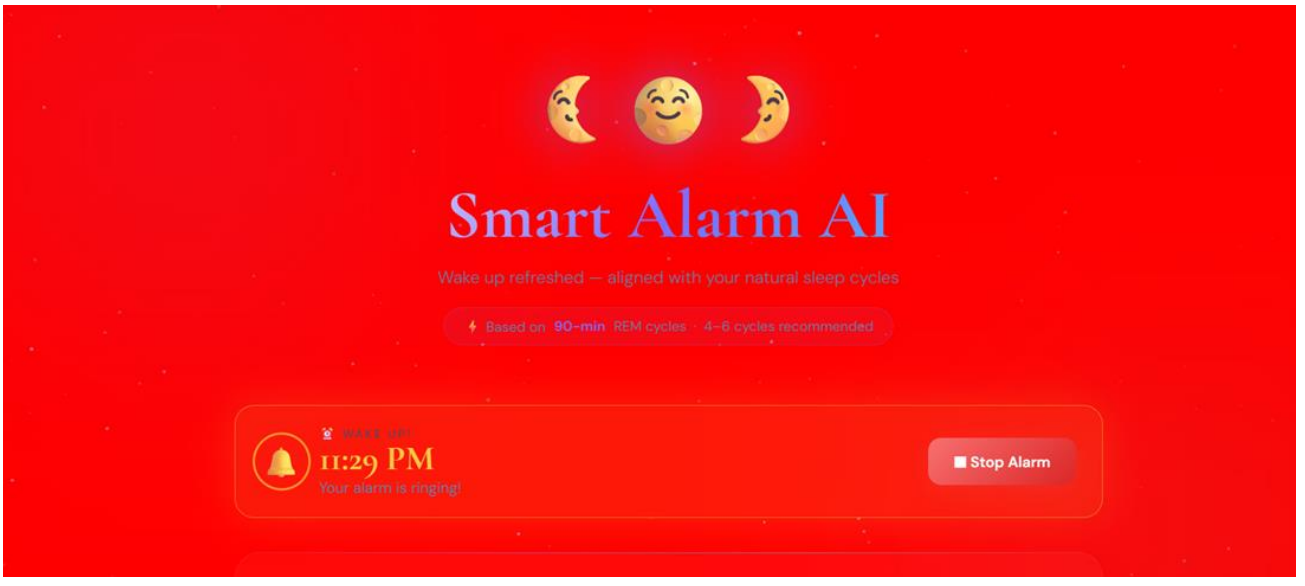

Fig:5.4. Alarm ringing

6.1 Conclusion:

The **Smart Alarm AI – Sleep Cycle Prediction System** successfully enhances the traditional alarm experience by integrating sleep science with intelligent time-based prediction. The system calculates optimal wake-up times using 90-minute sleep cycles, helping users wake up during lighter sleep stages and thereby improving alertness, productivity, and overall well-being.

Built using the Flask framework, the application provides a lightweight and efficient platform with a responsive user interface. Features such as sleep quality classification, percentage-based analysis, interactive alarm system, and history tracking make the system both practical and user-friendly.

6.2 Future Scope:

- ntegration of machine learning for personalized sleep cycle prediction
- Development of a mobile application for real-time alarm and notifications
- Integration with wearable devices for accurate sleep tracking
- Implementation of advanced alarm challenges like voice or logic-based tasks
- Addition of database support for storing user sleep history and analytics
- Smart notifications for bedtime reminders and daily productivity insights
- Integration with smart home systems and voice assistants

7.1 REFERENCES

- Python Flask Documentation – https://flask.palletsprojects.com/
  Guide for developing the backend web application, routing, and request handling.
- MDN Web Docs – https://developer.mozilla.org/
  Reference for frontend development using HTML, CSS, and JavaScript for building interactive user interfaces.
- Web Audio API Documentation – https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
  Used for implementing browser-based alarm sound generation and audio control.
- Sleep Cycle Research – https://www.sleepfoundation.org/how-sleep-works/sleep-cycles
  Provides scientific understanding of 90-minute sleep cycles and their impact on wake-up quality.
- JavaScript Timer Functions – https://developer.mozilla.org/en-US/docs/Web/API/setInterval
  Used for implementing real-time countdown and alarm triggering mechanisms.
- HTML5 Input Types – https://developer.mozilla.org/en-US/docs/Web/HTML/Element/input/time
  Reference for time-based input handling in the user interface.

7.2 **APPENDIX**

App.py:

```
from flask import Flask, render_template, request, redirect, jsonify
from datetime import datetime, timedelta

app = Flask(__name__)


# 🗃️ Memory Storage
history = []
MAX_HISTORY = 10

# □ Sleep Prediction Logic
def calculate_wake_times(sleep_time):
cycles = [4, 5, 6]
results = []

for c in cycles:
total_sleep = 90 * c
wake_time = sleep_time + timedelta(minutes=total_sleep)

percent = int((c / 6) * 100)

if c == 6:
quality = "Excellent 😍"
elif c == 5:
quality = "Good 🙂"
else:
quality = "Average 😐"

results.append({
"time": wake_time.strftime("%I:%M %p"),
"quality": quality,
"percent": percent
})

return results




# 🌐 Home Route
```

```
@app.route("/", methods=["GET", "POST"])
def home():
wake_times = None

if request.method == "POST":
sleep_input = request.form.get("sleep_time")

if not sleep_input:
return redirect("/")

try:
sleep_time = datetime.strptime(sleep_input, "%H:%M")

# ⏱ Fix past time → next day
now = datetime.now()

sleep_time = datetime. combine(now.date(), sleep_time.time())

if sleep_time < now:
sleep_time += timedelta(days=1)

except ValueError:
return redirect("/")

wake_times = calculate_wake_times(sleep_time)

# 📜 Save history
history.append({
"input": sleep_input,
"result": wake_times
})

if len(history) > MAX_HISTORY:
history.pop(0)

return render_template("index.html", wake_times=wake_times, history=history)

# 🧹 Clear History
@app.route("/clear")
def clear():
history.clear()
return redirect("/")
```

```
# 🚀 Run App
if __name__ == "__main__":
app.run(debug=True, use_reloader=False)
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Smart Alarm AI — Sleep Cycle Optimizer</title>
  <link
href="https://fonts.googleapis.com/css2?family=Cormorant+Garamond:wght@300;400;600;7
00&family=DM+Sans:wght@300;400;500;600&display=swap" rel="stylesheet">
  <style>
    :root {
      --bg-deep:       #03060f;
      --accent-violet: #7c6dff;
      --accent-cyan:   #38d9f5;
      --accent-gold:   #f5c842;
      --glass-bg:      rgba(255,255,255,0.035);
      --glass-border:  rgba(255,255,255,0.08);
      --text-primary:  #e8ecf8;
      --text-muted:    #6b7faa;
      --text-dim:      #3d4d72;
    }

    *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

    body {
      font-family: 'DM Sans', sans-serif;
      background: var(--bg-deep);
      color: var(--text-primary);
      min-height: 100vh;
      overflow-x: hidden;
    }

    #starfield { position: fixed; inset: 0; z-index: 0; pointer-events: none; }

    .nebula {
      position: fixed; border-radius: 50%;
      filter: blur(90px); pointer-events: none; z-index: 0;
    }
    .nebula-1 {
      width: 600px; height: 600px;
      background: radial-gradient(circle, rgba(124,109,255,0.13) 0%, transparent 70%);
      top: -200px; left: -150px;
      animation: drift 18s ease-in-out infinite alternate;
    }
    .nebula-2 {
```

```
  width: 500px; height: 500px;
  background: radial-gradient(circle, rgba(56,217,245,0.09) 0%, transparent 70%);
  bottom: -100px; right: -100px;
  animation: drift 22s ease-in-out infinite alternate-reverse;
}
.nebula-3 {
  width: 380px; height: 380px;
  background: radial-gradient(circle, rgba(244,114,182,0.07) 0%, transparent 70%);
  top: 40%; left: 55%;
  animation: drift 26s ease-in-out infinite alternate;
}
@keyframes drift {
  from { transform: translate(0,0) scale(1); }
  to   { transform: translate(40px,30px) scale(1.08); }
}
@keyframes float {
  0%,100% { transform: translateY(0); }
  50%     { transform: translateY(-10px); }
}
@keyframes fadeUp {
  from { opacity: 0; transform: translateY(24px); }
  to   { opacity: 1; transform: translateY(0); }
}
@keyframes pulse-ring {
  0%   { transform: scale(1);   opacity: 0.8; }
  100% { transform: scale(1.6); opacity: 0; }
}

.page-wrap {
  position: relative; z-index: 1;
  max-width: 860px; margin: 0 auto; padding: 60px 24px 100px;
}

/* ── ALARM BANNER ──────────────────────────────────────────────── */
#alarm-banner {
  display: none;
  background: rgba(124,109,255,0.1);
  border: 1px solid rgba(124,109,255,0.3);
  border-radius: 18px; padding: 18px 24px;
  margin-bottom: 32px;
  backdrop-filter: blur(16px); -webkit-backdrop-filter: blur(16px);
  animation: fadeUp 0.4s ease both;
}
#alarm-banner.firing {
  background: rgba(245,200,66,0.12);
  border-color: rgba(245,200,66,0.5);
  box-shadow: 0 0 40px rgba(245,200,66,0.2);
}
.banner-inner { display: flex; align-items: center; gap: 16px; flex-wrap: wrap; }
.banner-icon { font-size: 28px; position: relative; }
.banner-icon .ring {
  position: absolute; inset: -6px; border-radius: 50%;
  border: 2px solid var(--accent-violet); opacity: 0;
}
#alarm-banner.firing .banner-icon .ring {
  animation: pulse-ring 0.9s ease-out infinite;
  border-color: var(--accent-gold);
}
.banner-text { flex: 1; }
```

```
    .banner-label { font-size: 11px; text-transform: uppercase; letter-spacing: 2px; color: var(--
text-dim); }
    .banner-time  { font-family: 'Cormorant Garamond', serif; font-size: 32px; font-weight: 700;
color: var(--accent-violet); line-height: 1; }
    #alarm-banner.firing .banner-time { color: var(--accent-gold); }
    .banner-countdown { font-size: 14px; color: var(--text-muted); margin-top: 2px; }

    .stop-btn {
      padding: 12px 24px;
      background: linear-gradient(135deg, #f87171, #dc2626);
      border: none; border-radius: 12px; color: white;
      font-family: 'DM Sans', sans-serif; font-size: 14px; font-weight: 600;
      cursor: pointer; display: none;
      transition: transform 0.2s, box-shadow 0.2s;
      box-shadow: 0 4px 20px rgba(239,68,68,0.35);
    }
    .stop-btn:hover { transform: translateY(-2px); box-shadow: 0 6px 28px rgba(239,68,68,0.5);
}

    .cancel-btn {
      padding: 10px 18px;
      background: rgba(255,255,255,0.05);
      border: 1px solid rgba(255,255,255,0.1); border-radius: 12px; color: var(--text-muted);
      font-family: 'DM Sans', sans-serif; font-size: 13px; font-weight: 500;
      cursor: pointer; transition: background 0.2s;
    }
    .cancel-btn:hover { background: rgba(255,255,255,0.1); }

    /* ── HERO
──────────────────────────────────────────────────────────────────────── */
    .hero { text-align: center; margin-bottom: 56px; }

    .moon-icon {
      display: inline-block; font-size: 64px; margin-bottom: 16px;
      animation: float 4s ease-in-out infinite;
      filter: drop-shadow(0 0 24px rgba(124,109,255,0.7));
    }

    .hero h1 {
      font-family: 'Cormorant Garamond', serif;
      font-size: clamp(38px, 7vw, 64px); font-weight: 700; line-height: 1.1;
      background: linear-gradient(135deg, #fff 0%, var(--accent-violet) 50%, var(--accent-cyan)
100%);
      -webkit-background-clip: text; -webkit-text-fill-color: transparent; background-clip: text;
    }

    .hero .tagline { margin-top: 12px; font-size: 16px; color: var(--text-muted); font-weight:
300; }

    .cycle-info {
      display: inline-flex; align-items: center; gap: 8px;
      margin-top: 18px; padding: 8px 18px; border-radius: 999px;
      border: 1px solid rgba(124,109,255,0.25); background: rgba(124,109,255,0.08);
      font-size: 13px; color: var(--text-muted);
    }
    .cycle-info span { color: var(--accent-violet); font-weight: 600; }

    /* ── FORM
──────────────────────────────────────────────────────────────────────── */
    .form-card {
```

```
    background: var(--glass-bg); border: 1px solid var(--glass-border);
    border-radius: 24px; padding: 40px;
    backdrop-filter: blur(20px); -webkit-backdrop-filter: blur(20px);
    margin-bottom: 48px; position: relative; overflow: hidden;
  }
  .form-card::before {
    content: ''; position: absolute; top: 0; left: 0; right: 0; height: 1px;
    background: linear-gradient(90deg, transparent, rgba(124,109,255,0.5),
rgba(56,217,245,0.4), transparent);
  }

  .form-label {
    display: block; font-size: 12px; text-transform: uppercase;
    letter-spacing: 2px; color: var(--text-muted); margin-bottom: 16px; font-weight: 500;
  }

  .input-row { display: flex; gap: 12px; align-items: stretch; }

  input[type="time"] {
    flex: 1; padding: 16px 20px;
    background: rgba(255,255,255,0.04); border: 1px solid rgba(255,255,255,0.1);
    border-radius: 14px; color: white; font-size: 20px;
    font-family: 'DM Sans', sans-serif; font-weight: 500; outline: none;
    transition: border-color 0.3s, box-shadow 0.3s; color-scheme: dark;
  }
  input[type="time"]:focus {
    border-color: var(--accent-violet);
    box-shadow: 0 0 0 3px rgba(124,109,255,0.15), 0 0 20px rgba(124,109,255,0.1);
  }

  .predict-btn {
    padding: 16px 32px;
    background: linear-gradient(135deg, var(--accent-violet), #5b4fcf);
    border: none; border-radius: 14px; color: white;
    font-family: 'DM Sans', sans-serif; font-size: 15px; font-weight: 600;
    cursor: pointer; white-space: nowrap; position: relative; overflow: hidden;
    transition: transform 0.2s, box-shadow 0.2s;
    box-shadow: 0 4px 24px rgba(124,109,255,0.35);
  }
  .predict-btn::before {
    content: ''; position: absolute; inset: 0;
    background: linear-gradient(135deg, rgba(255,255,255,0.15), transparent);
    opacity: 0; transition: opacity 0.2s;
  }
  .predict-btn:hover { transform: translateY(-2px); box-shadow: 0 8px 32px
rgba(124,109,255,0.5); }
  .predict-btn:hover::before { opacity: 1; }
  .predict-btn:active { transform: translateY(0); }

  .hint-text { margin-top: 16px; font-size: 13px; color: var(--text-dim); }

  /* ── RESULTS
──────────────────────────────────────────────────────────────── */
  .section-title {
    font-family: 'Cormorant Garamond', serif; font-size: 28px; font-weight: 600;
    margin-bottom: 24px; display: flex; align-items: center; gap: 10px;
  }
  .section-title .line {
    flex: 1; height: 1px;
    background: linear-gradient(90deg, var(--glass-border), transparent);
```

```
  }

  .results-grid { display: grid; grid-template-columns: repeat(3,1fr); gap: 18px; margin-bottom: 56px; }
  @media (max-width: 640px) { .results-grid { grid-template-columns: 1fr; } }

  .result-card {
    background: var(--glass-bg); border: 1px solid var(--glass-border);
    border-radius: 20px; padding: 28px 24px;
    backdrop-filter: blur(16px); -webkit-backdrop-filter: blur(16px);
    position: relative; overflow: hidden;
    animation: fadeUp 0.5s ease both; transition: transform 0.3s, box-shadow 0.3s;
  }
  .result-card:nth-child(1) { animation-delay: 0.05s; }
  .result-card:nth-child(2) { animation-delay: 0.15s; }
  .result-card:nth-child(3) { animation-delay: 0.25s; }
  .result-card:hover { transform: translateY(-4px); }
  .result-card::before { content: ''; position: absolute; top: 0; left: 0; right: 0; height: 2px; }

  .quality-excellent { border-color: rgba(245,200,66,0.3); box-shadow: 0 0 32px rgba(245,200,66,0.07); }
  .quality-excellent:hover { box-shadow: 0 8px 40px rgba(245,200,66,0.15); }
  .quality-excellent::before { background: linear-gradient(90deg, transparent, var(--accent-gold), transparent); }
  .quality-excellent .card-badge { background: rgba(245,200,66,0.12); color: var(--accent-gold); }
  .quality-excellent .card-time  { color: var(--accent-gold); }
  .quality-excellent .prog-bar   { background: linear-gradient(90deg, #f5c842, #fde68a); }
  .quality-excellent .alarm-btn  { border-color: rgba(245,200,66,0.35); color: var(--accent-gold); }
  .quality-excellent .alarm-btn:hover { background: rgba(245,200,66,0.12); }

  .quality-good { border-color: rgba(56,217,245,0.25); box-shadow: 0 0 32px rgba(56,217,245,0.05); }
  .quality-good:hover { box-shadow: 0 8px 40px rgba(56,217,245,0.12); }
  .quality-good::before { background: linear-gradient(90deg, transparent, var(--accent-cyan), transparent); }
  .quality-good .card-badge { background: rgba(56,217,245,0.10); color: var(--accent-cyan); }
  .quality-good .card-time  { color: var(--accent-cyan); }
  .quality-good .prog-bar   { background: linear-gradient(90deg, #38d9f5, #a5f3fc); }
  .quality-good .alarm-btn  { border-color: rgba(56,217,245,0.35); color: var(--accent-cyan); }
  .quality-good .alarm-btn:hover { background: rgba(56,217,245,0.1); }

  .quality-average::before { background: linear-gradient(90deg, transparent, #94a3b8, transparent); }
  .quality-average .card-badge { background: rgba(148,163,184,0.1); color: #94a3b8; }
  .quality-average .card-time  { color: #cbd5e1; }
  .quality-average .prog-bar   { background: linear-gradient(90deg, #64748b, #94a3b8); }
  .quality-average .alarm-btn  { border-color: rgba(148,163,184,0.25); color: #94a3b8; }
  .quality-average .alarm-btn:hover { background: rgba(148,163,184,0.08); }

  .card-badge {
    display: inline-block; font-size: 10px; font-weight: 600;
    text-transform: uppercase; letter-spacing: 1.5px;
    padding: 4px 10px; border-radius: 999px; margin-bottom: 16px;
  }
  .card-time {
    font-family: 'Cormorant Garamond', serif; font-size: 42px; font-weight: 700;
    letter-spacing: -1px; line-height: 1; margin-bottom: 6px;
  }
```

```
  .card-quality { font-size: 14px; color: var(--text-muted); margin-bottom: 20px; }

  .prog-wrap {
    background: rgba(255,255,255,0.05); border-radius: 999px;
    height: 6px; overflow: hidden; margin-bottom: 8px;
  }
  .prog-bar { height: 100%; border-radius: 999px; width: 0; transition: width 1.2s cubic-bezier(0.4,0,0.2,1); }

  .prog-label { font-size: 12px; color: var(--text-dim); text-align: right; }
  .prog-label strong { color: var(--text-muted); font-weight: 600; }

  .cycles-tag { margin-top: 16px; display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text-dim); }
  .cycles-tag .dot { width: 5px; height: 5px; border-radius: 50%; background: currentColor; opacity: 0.5; }

  /* ── ALARM BUTTON ──────────────────────────────────────────────── */
  .alarm-btn {
    margin-top: 18px; width: 100%;
    padding: 11px; border-radius: 12px;
    background: transparent; border: 1px solid rgba(255,255,255,0.12);
    font-family: 'DM Sans', sans-serif; font-size: 13px; font-weight: 500;
    color: var(--text-muted); cursor: pointer;
    transition: background 0.2s, border-color 0.2s, color 0.2s;
    display: flex; align-items: center; justify-content: center; gap: 8px;
  }
  .alarm-btn.active {
    background: rgba(124,109,255,0.15) !important;
    border-color: var(--accent-violet) !important;
    color: white !important;
  }

  /* ── HISTORY ──────────────────────────────────────────────── */
  .history-controls { display: flex; align-items: center; justify-content: space-between; margin-bottom: 20px; }

  .clear-btn {
    display: inline-flex; align-items: center; gap: 6px;
    padding: 9px 18px; background: rgba(239,68,68,0.1);
    border: 1px solid rgba(239,68,68,0.2); border-radius: 10px; color: #f87171;
    font-family: 'DM Sans', sans-serif; font-size: 13px; font-weight: 500; cursor: pointer;
    transition: background 0.2s, border-color 0.2s;
  }
  .clear-btn:hover { background: rgba(239,68,68,0.18); border-color: rgba(239,68,68,0.4); }

  .history-list { display: flex; flex-direction: column; gap: 14px; }

  .history-card {
    background: var(--glass-bg); border: 1px solid var(--glass-border);
    border-radius: 16px; padding: 20px 24px;
    backdrop-filter: blur(12px); -webkit-backdrop-filter: blur(12px);
    display: flex; align-items: flex-start; gap: 20px;
    animation: fadeUp 0.4s ease both; transition: border-color 0.25s;
  }
  .history-card:hover { border-color: rgba(124,109,255,0.2); }

  .history-time-badge {
```

```
      flex-shrink: 0; background: rgba(124,109,255,0.1);
      border: 1px solid rgba(124,109,255,0.2); border-radius: 12px;
      padding: 10px 14px; text-align: center;
    }
    .history-time-badge .slept-label { font-size: 10px; text-transform: uppercase; letter-spacing:
1px; color: var(--text-dim); }
    .history-time-badge .slept-time {
      font-family: 'Cormorant Garamond', serif; font-size: 22px; font-weight: 700;
      color: var(--accent-violet); line-height: 1.2;
    }

    .history-results { display: flex; flex-wrap: wrap; gap: 10px; flex: 1; align-items: center; }

    .history-chip {
      display: inline-flex; align-items: center; gap: 6px;
      padding: 6px 12px; border-radius: 8px; font-size: 13px; font-weight: 500;
      background: rgba(255,255,255,0.04); border: 1px solid rgba(255,255,255,0.07);
    }
    .chip-pct { color: var(--text-dim); font-size: 11px; }

    .footer {
      text-align: center; margin-top: 80px; padding-top: 32px;
      border-top: 1px solid var(--glass-border); color: var(--text-dim); font-size: 13px;
    }
  </style>
</head>
<body>

<canvas id="starfield"></canvas>
<div class="nebula nebula-1"></div>
<div class="nebula nebula-2"></div>
<div class="nebula nebula-3"></div>

<div class="page-wrap">

  <header class="hero">
    <div class="moon-icon">🌛</div>
    <h1>Smart Alarm AI</h1>
    <p class="tagline">Wake up refreshed — aligned with your natural sleep cycles</p>
    <div class="cycle-info">
      ⚡ Based on <span>90-min</span> REM cycles &amp;nbsp;·&amp;nbsp; 4–6 cycles recommended
    </div>
  </header>

  <!-- ── ALARM STATUS BANNER
──────────────────────────────────────────────────────────── -->
  <div id="alarm-banner">
    <div class="banner-inner">
      <div class="banner-icon">
        🔔
        <div class="ring"></div>
      </div>
      <div class="banner-text">
        <div class="banner-label" id="banner-label">Alarm set for</div>
        <div class="banner-time"  id="banner-time">—</div>
        <div class="banner-countdown" id="banner-countdown"></div>
      </div>
      <button class="stop-btn"   id="stop-btn"   onclick="stopAlarm()">■ Stop Alarm</button>
      <button class="cancel-btn" id="cancel-btn" onclick="cancelAlarm()">✕ Cancel</button>
    </div>
```

```
  </div>

  <!-- ── FORM ──────────────────────────────────────────────────────────── -->
  <div class="form-card">
    <label class="form-label">⏱ What time are you sleeping?</label>
    <form method="POST">
      <div class="input-row">
        <input type="time" name="sleep_time" required />
        <button type="submit" class="predict-btn">✦ Predict Wake Times</button>
      </div>
      <p class="hint-text">💡 Enter your bedtime and we'll find your perfect wake-up windows</p>
    </form>
  </div>

  {% if wake_times %}
  <section>
    <div class="section-title">⏰ Your Sleep Windows <div class="line"></div></div>
    <div class="results-grid">
      {% for item in wake_times %}
        {% if "Excellent" in item.quality %}
          {% set cls = "quality-excellent" %}
          {% set badge = "✦ Best Choice" %}
          {% set cycles_n = 6 %}
        {% elif "Good" in item.quality %}
          {% set cls = "quality-good" %}
          {% set badge = "● Recommended" %}
          {% set cycles_n = 5 %}
        {% else %}
          {% set cls = "quality-average" %}
          {% set badge = "○ Acceptable" %}
          {% set cycles_n = 4 %}
        {% endif %}

        <div class="result-card {{ cls }}">
          <div class="card-badge">{{ badge }}</div>
          <div class="card-time">{{ item.time }}</div>
          <div class="card-quality">{{ item.quality }}</div>
          <div class="prog-wrap">
            <div class="prog-bar" data-width="{{ item.percent }}"></div>
          </div>
          <div class="prog-label"><strong>{{ item.percent }}%</strong> sleep quality</div>
          <div class="cycles-tag">
            <div class="dot"></div>
            {{ cycles_n }} cycles · {{ (cycles_n * 90) // 60 }}h {{ (cycles_n * 90) % 60 }}m total rest
          </div>

          <!-- SET ALARM BUTTON -->
          <button class="alarm-btn" data-time="{{ item.time }}" onclick="setAlarm(this)">
            🔔 Set Alarm for {{ item.time }}
          </button>
        </div>
      {% endfor %}
    </div>
  </section>
  {% endif %}
```

```
  {% if history %}
  <section class="history-section">
    <div class="history-controls">
      <div class="section-title" style="margin-bottom:0;">📜 History <div
class="line"></div></div>
      <form action="/clear" method="GET" style="margin:0;">
        <button class="clear-btn" type="submit">🗑 Clear All</button>
      </form>
    </div>
    <div class="history-list">
      {% for item in history | reverse %}
      <div class="history-card">
        <div class="history-time-badge">
          <div class="slept-label">Slept at</div>
          <div class="slept-time">{{ item.input }}</div>
        </div>
        <div class="history-results">
          {% for r in item.result %}
            {% if "Excellent" in r.quality %}{% set dot = "🟩" %}
            {% elif "Good" in r.quality %}{% set dot = "🟨" %}
            {% else %}{% set dot = "🟥" %}{% endif %}
            <div class="history-chip">
              <span>{{ dot }}</span>
              <span>{{ r.time }}</span>
              <span class="chip-pct">{{ r.percent }}%</span>
            </div>
          {% endfor %}
        </div>
      </div>
      {% endfor %}
    </div>
  </section>
  {% endif %}

  <footer class="footer">
    Smart Alarm AI — Built on sleep science &amp;amp; 90-minute REM cycles
  </footer>
</div>

<!-- ═══════════════════════════ JAVASCRIPT
═══════════════════════════ -->
<script>
/* ── Starfield
──────────────────────────────────────────────────────────────── */

(function () {

  const canvas = document.getElementById('starfield');
  const ctx = canvas.getContext('2d');
  let W, H, stars = [];
  function resize() { W = canvas.width = window.innerWidth; H = canvas.height =
window.innerHeight; }
  function mkStar() {
    return { x: Math.random()*W, y: Math.random()*H, r: Math.random()*1.3+0.2,
             a: Math.random(), da: (Math.random()-0.5)*0.004, spd: Math.random()*0.06+0.01 };
  }
  function init() { resize(); stars = Array.from({ length: 200 }, mkStar); }
  function draw() {
    ctx.clearRect(0, 0, W, H);
    stars.forEach(s => {
```

```
      s.y += s.spd; s.a += s.da;
      if (s.a < 0) { s.a = 0; s.da *= -1; }
      if (s.a > 1) { s.a = 1; s.da *= -1; }
      if (s.y > H) { s.y = -2; s.x = Math.random()*W; }
      ctx.beginPath();
      ctx.arc(s.x, s.y, s.r, 0, Math.PI*2);
      ctx.fillStyle = `rgba(200,210,255,${s.a * 0.7})`;
      ctx.fill();
    });
    requestAnimationFrame(draw);
  }
  init(); draw();
  window.addEventListener('resize', resize);
})();

/* ── Progress bars ──────────────────────────────────────────────────────── */
document.addEventListener('DOMContentLoaded', () => {
  setTimeout(() => {
    document.querySelectorAll('.prog-bar').forEach(bar => {
      bar.style.width = bar.getAttribute('data-width') + '%';
    });
  }, 120);
});

/* ── Alarm Engine ──────────────────────────────────────────────────────── */
let audioCtx       = null;
let alarmTarget    = null;   // { h, m } in 24-h
let alarmLabel     = '';
let alarmInterval  = null;
let alarmNode      = null;   // currently playing sound source
let isFiring       = false;

// Parse "hh:mm AM/PM" → { h24, m }
function parse12h(str) {
  const m = str.match(/(\d+):(\d+)\s*(AM|PM)/i);
  if (!m) return null;
  let h = parseInt(m[1], 10);
  const min = parseInt(m[2], 10);
  const pm = m[3].toUpperCase() === 'PM';
  if (pm && h !== 12) h += 12;
  if (!pm && h === 12) h = 0;
  return { h, m: min };
}

// Lazy-init Web Audio context (browsers need user gesture)
function getAudioCtx() {
  if (!audioCtx) audioCtx = new (window.AudioContext || window.webkitAudioContext)();
  return audioCtx;
}

/*
  Classic two-tone alarm: alternates between 880 Hz and 1100 Hz
  in short bursts (beep-beep pattern) using OscillatorNode + GainNode.
*/
function playAlarmSound() {
  const ctx = getAudioCtx();
  alarmNode = { stop: () => {} }; // placeholder
```

```
  let running = true;
  const master = ctx.createGain();
  master.gain.value = 0.55;
  master.connect(ctx.destination);

  // Store stop handle on the alarmNode reference
  alarmNode = { stop: () => { running = false; master.disconnect(); } };

  const freqs  = [880, 1100];
  const onMs   = 140;   // beep duration
  const offMs  = 80;    // gap between beeps
  const pauseMs = 420;  // pause after each pair

  let step = 0;

  function nextBeep() {
    if (!running) return;

    const freq = freqs[step % 2];
    const osc  = ctx.createOscillator();
    const env  = ctx.createGain();

    osc.type      = 'square';
    osc.frequency.value = freq;

    // Quick attack + decay envelope so it doesn't clip
    const now = ctx.currentTime;
    env.gain.setValueAtTime(0, now);
    env.gain.linearRampToValueAtTime(1, now + 0.01);
    env.gain.linearRampToValueAtTime(0, now + onMs / 1000 - 0.01);

    osc.connect(env);
    env.connect(master);

    osc.start(now);
    osc.stop(now + onMs / 1000);

    step++;
    const gap = (step % 2 === 0) ? pauseMs : offMs;
    setTimeout(nextBeep, onMs + gap);
  }

  nextBeep();
}

function stopAlarmSound() {
  if (alarmNode) { alarmNode.stop(); alarmNode = null; }
}

/* ── UI helpers ──────────────────────────────────────────────────────────── */
function formatCountdown(diffMs) {
  if (diffMs <= 0) return 'Ringing now!';
  const totalSec = Math.round(diffMs / 1000);
  const h = Math.floor(totalSec / 3600);
  const m = Math.floor((totalSec % 3600) / 60);
  const s = totalSec % 60;
  const parts = [];
  if (h) parts.push(h + 'h');
  if (m) parts.push(m + 'm');
```

```
  parts.push(s + 's');
  return `Rings in ${parts.join (' ')}`;
}

function showBanner(label) {
  const banner = document.getElementById('alarm-banner');
  document.getElementById('banner-time').textContent = label;
  document.getElementById('banner-label').textContent = 'Alarm set for';
  banner.classList.remove('firing');
  banner.style.display = 'block';
  document.getElementById('stop-btn').style.display   = 'none';
  document.getElementById('cancel-btn').style.display = 'inline-flex';
}

function triggerFiring() {
  const banner = document.getElementById('alarm-banner');
  banner.classList.add('firing');
  document.getElementById('banner-label').textContent    = '⏰ Wake up!';
  document.getElementById('banner-countdown').textContent = 'Your alarm is ringing!';
  document.getElementById('stop-btn').style.display      = 'inline-flex';
  document.getElementById('cancel-btn').style.display    = 'none';
  document.body.style.background = "red";
  document.title = "WAKE UP!!! 🚨";
  isFiring = true;
  playAlarmSound();
  // Vibrate on mobile if supported
  if (navigator.vibrate) navigator.vibrate([400,200,400,200,400]);
}

/* ── Set / cancel / stop ──────────────────────────────────────────── */
window.setAlarm = function(btn) {
  const timeStr = btn.getAttribute('data-time');
  const parsed  = parse12h(timeStr);
  if (!parsed) return;

  // Clear previous
  cancelAlarm(true);

  alarmTarget = parsed;
  alarmLabel  = timeStr;

  // Mark button active, unmark others
  document.querySelectorAll('.alarm-btn').forEach(b => b.classList.remove('active'));
  btn.classList.add('active');
  btn.innerHTML = '✓ Alarm set for ' + timeStr;
  alert("⚠ Make sure your device is NOT in silent mode and volume is high.");
  showBanner(timeStr);
  startCountdownTick();
};

window.cancelAlarm = function(silent) {
  clearInterval(alarmInterval);
  stopAlarmSound();
  alarmTarget = null;
  isFiring    = false;
  document.querySelectorAll('.alarm-btn').forEach(b => {
    b.classList.remove('active');
    const t = b.getAttribute('data-time');
    b.innerHTML = '🔔 Set Alarm for ' + t;
```

```
  });
  if (!silent) {
    document.getElementById('alarm-banner').style.display = 'none';
  }
};

window.stopAlarm = function() {
    const answer = prompt("Solve this to stop alarm: 7 + 5 = ?");

    if (answer == "12") {
        stopAlarmSound();
        isFiring = false;

        const banner = document.getElementById('alarm-banner');
        banner.classList.remove('firing');

        document.getElementById('banner-label').textContent = 'Alarm stopped';
        document.getElementById('banner-countdown').textContent = 'Have a great morning!
☼';

        document.getElementById('stop-btn').style.display = 'none';
        document.getElementById('cancel-btn').style.display = 'inline-flex';

        clearInterval(alarmInterval);

        document.querySelectorAll('.alarm-btn').forEach(b => {
            b.classList.remove('active');
            b.innerHTML = '🔔 Set Alarm for ' + b.getAttribute('data-time');
        });

    } else {
        alert("Wrong answer! Try again 😈");
    }
};
function startCountdownTick() {
  clearInterval(alarmInterval);
  alarmInterval = setInterval(() => {
    if (!alarmTarget || isFiring) return;

    const now   = new Date();
    let target  = new Date();
    target.setHours(alarmTarget.h, alarmTarget.m, 0, 0);

    // If target is in the past, push to tomorrow
    if (target <= now) target.setDate(target.getDate() + 1);

    const diff = target - now;

    // Fire when within 1 second
    if (diff < 1000) {
      clearInterval(alarmInterval);
      triggerFiring();
      return;
    }

    document.getElementById('banner-countdown').textContent = formatCountdown(diff);
  }, 1000);
}
</script></body></html>
```

PROJECT TITLE	**AI SMART ALARM**	
PROGRAM	**B.Tech. ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**	
PROJECT NUMBER		
REGISTER NUMBER & NAME	**722825243127**	**NAVEEN KUMAR M**
NAME OF THE SUPERVISOR	**Mr.R.Divakar, Assistant Professor, AI&DS Department**	
NAME OF THE SDG GOALS MAPPED	**Good Health and Well-being**	
MENTION THE SDG GOALS NUMBER	**SDG 3**	
NAME OF THE TRL LEVEL	**Prototype Demonstration in a Relevant Environment**	
MENTION THE TRL LEVEL	**TRL - 4**	

POs & PSOs Mapping (Put a tick mark in the mapped PO's & PSO's):

CO	PO 01	PO 02	PO 03	PO 04	PO 05	PO 06	PO 07	PO 08	PO 09	PO 10	PO 11	PSO 01	PSO 02
U23AD491.1	2	2	3	2	3	-	-	3	-	-	3	2	1
U23AD491.2	2	2	3	2	3	-	-	3	-	-	3	2	1
U23AD491.3	2	2	3	2	3	-	-	3	-	-	3	2	1
U23AD491.4	2	2	3	2	3	-	-	3	-	-	3	2	1
U23AD491.5	2	2	3	2	3	-	-	3	-	-	3	2	1
Average	2	2	3	2	3	-	-	3	-	-	3	2	1

Signature of the Supervisor

(Mr.R.Divakar M.E., (Ph.D))

VENUE AND EXPENDITURE STATEMENT FOR THE PROJECT WORK

Laboratory details where the project is carried out	Artificial Intelligence Lab, AI BLOCK
Software Details	• **Operating System:** Windows 10/11 or Linux/macOS • **IDE:** Visual Studio Code • **Backend:** Python 3.x with Flask Framework • **Frontend:** HTML, CSS, JavaScript • **Libraries/Technologies:** • datetime (time calculations) • Web Audio API (alarm sound generation) • JavaScript (countdown & interaction logic)

Details of the Component and Expenditure

S. No	Name of the Component	Qty	Price / Unit in (Rs.)	Amount (Rs.)

(*Include any other charges which includes fabrication cost and others)

Signature of the student
(NAVEEN KUMAR M)

Signature of the Supervisor
Mr. R. Divakar